

Forense de red: capturas, sesiones e indicadores

Del sistema operativo a la red

En la Lección 07 se reconstruyó una secuencia a partir de lo que **el propio equipo** registró: eventos, cuentas, procesos y tareas. Esa evidencia tiene una debilidad conocida: la escribe el mismo sistema que se investiga, y depende de su configuración de auditoría.

La red ofrece un punto de vista distinto. Un sensor externo al equipo registra lo que efectivamente viajó por el cable, con su propio reloj, sin depender de lo que el equipo decidió anotar. La pregunta cambia:

¿Qué comunicaciones pueden demostrarse desde lo que pasó por la red, entre quiénes, en qué orden, y qué queda fuera del alcance de lo que un sensor puede ver?

Tres ideas ordenan la lección:

1. una captura muestra **comunicaciones entre direcciones**, no acciones de personas ni de programas;
2. cada afirmación se sostiene con un **número de paquete** o un **filtro reproducible**, no con una captura de pantalla sin contexto;
3. un indicador vale por su **contexto**: fuente, ventana de tiempo y explicación alternativa.

1. Qué es evidencia de red y de dónde proviene

No toda la evidencia de red es una captura completa. Existen tres familias, con costos y alcances distintos:

Tipo	Qué conserva	Ejemplos	Límite principal
Captura completa de paquetes	Cada paquete con cabeceras y contenido	Archivos <code>pcap</code> o <code>pcapng</code> de <code>tcpdump</code> , Wireshark o un sensor	Ocupa mucho espacio: suele conservarse por horas o días
Registros de flujo	Un resumen por conexión: direcciones, puertos, bytes, duración	NetFlow, IPFIX, tablas de conexiones de un firewall	No contiene el contenido; no permite reconstruir archivos
Registros de aplicaciones de red	Lo que un servicio decidió anotar	Proxy web, resolutor DNS, servidor de correo, Zeek	Solo ve el tráfico que pasa por ese servicio y en su formato

En la práctica se combinan: el flujo muestra que existió una conexión durante meses; la captura completa permite reconstruir qué se transmitió en la ventana en que estuvo disponible. En la Evaluación UA2 se trabajará con registros de proxy, que pertenecen a la tercera familia.

El punto de captura decide lo que se ve

Un sensor solo registra lo que pasa por donde está conectado.

Punto de captura	Qué ve	Qué no ve
Puerto SPAN (<i>port mirroring</i>) de un switch	Tráfico de los puertos copiados hacia el sensor	Tráfico de otros switches; paquetes que el switch descarta si se satura
TAP de red	Copia física exacta de un enlace	Cualquier otro enlace
Captura en el propio equipo	Tráfico de ese equipo, incluso el cifrado localmente antes de salir	Tráfico de otros equipos; puede ser alterada si el equipo está comprometido
Perímetro (salida a Internet)	Comunicaciones hacia el exterior, a menudo después de NAT	Tráfico interno entre equipos; la dirección interna real si hay NAT

Por eso la ficha técnica de una captura debe declarar dónde se tomó. «No se observa tráfico hacia X» solo significa «no pasó por este sensor durante esta ventana».

2. Capas, direcciones y puertos

Cada paquete lleva información de varias capas. Para el análisis forense bastan cuatro:

Capa	Dato	Qué identifica	Advertencia
Enlace (Ethernet)	Dirección MAC	Una interfaz de red dentro del mismo segmento	Se reemplaza en cada salto: fuera de la red local se ve la MAC del router, no la del equipo remoto

Capa	Dato	Qué identifica	Advertencia
Red (IP)	Dirección IP	Un extremo lógico de la comunicación	Puede ser compartida, traducida por NAT o asignada dinámicamente
Transporte (TCP o UDP)	Puertos de origen y destino	El servicio y la conexión	Un puerto es una convención, no una garantía del protocolo real
Aplicación	DNS, HTTP, TLS y otros	El contenido o la negociación del servicio	Puede ir cifrado

La MAC y el fabricante

Los tres primeros bytes de una dirección MAC forman el **OUI**, un prefijo que el IEEE asigna a cada fabricante. Wireshark lo traduce a un nombre. El prefijo `08:00:27` , por ejemplo, está asignado a PCS Systemtechnik GmbH, y es el que usa VirtualBox para las interfaces de sus máquinas virtuales.

El dato sirve como pista, no como identificación: la MAC puede cambiarse por software, algunas están marcadas como administradas localmente y ninguna identifica a una persona. Además, solo es visible dentro del segmento donde se capturó.

La 5-tupla

Una conexión se identifica por cinco valores: IP de origen, puerto de origen, IP de destino, puerto de destino y protocolo. El cliente elige un puerto de origen alto y temporal, llamado **efímero**; el servidor escucha en un puerto conocido, como `53` para DNS, `80` para HTTP o `443` para HTTPS. Dos conexiones sucesivas entre los mismos equipos se distinguen por el puerto efímero.

3. El archivo de captura

Aspecto	Qué registrar	Por qué importa
Formato	<code>pcap</code> o <code>pcapng</code>	<code>pcapng</code> puede incluir varias interfaces y comentarios; ambos se analizan con las mismas herramientas
<i>Snaplen</i> (longitud de captura)	Bytes guardados por paquete	Si es corta, los paquetes quedan truncados y no se puede reconstruir el contenido
Encapsulación	Ethernet u otra	Determina cómo se interpreta cada paquete
Primer y último paquete	Horas en UTC	Delimitan la ventana de tráfico observado; no prueban cuándo se detuvo la captura
Hash	SHA-256 del archivo	Comprueba que se analiza la copia entregada

El tiempo en una captura

Cada paquete guarda un *timestamp* tomado por el **reloj del sensor** en el momento de recibirlo, almacenada como segundos desde 1970 en UTC. Dos consecuencias:

- **La hora de la captura es la del sensor, no la del equipo investigado.** Si el sensor estaba desfasado, todos los paquetes lo están en la misma cantidad; si el equipo estaba desfasado, sus propios registros no coincidirán con la captura.
- **La hora que muestra Wireshark depende de quien la abre.** Por defecto se presenta en la zona horaria del equipo de análisis. Dos analistas en zonas distintas verán horas distintas para el mismo paquete. Por eso se trabaja en UTC y se normaliza a la zona del caso de forma explícita, como en la Lección 07.

4. TCP: conexiones, secuencia y cierre

TCP establece una conexión antes de transmitir datos, mediante el ***three-way handshake***:

cliente -> servidor	SYN	quiero abrir una conexión
servidor -> cliente	SYN, ACK	acepto
cliente -> servidor	ACK	conexión establecida

Luego se transmiten datos, cada uno confirmado con `ACK` , y la conexión termina con un intercambio de `FIN` o se interrumpe con `RST` .

Señal	Lectura forense
<code>SYN</code> sin respuesta	Se intentó una conexión; no hay evidencia de que se estableciera

Señal	Lectura forense
<code>SYN</code> seguido de <code>RST</code>	El destino rechazó la conexión: puerto cerrado o filtrado
<i>Handshake</i> completo y datos	Hubo comunicación efectiva entre ambos extremos
<code>FIN</code> de ambos lados	Cierre ordenado de la conexión

Números de secuencia y `tcp.stream`

Cada extremo numera los bytes que envía. Wireshark muestra esos números como **relativos**, comenzando en `0` , para facilitar la lectura. También asigna a cada conexión un índice, `tcp.stream` , que empieza en `0` para la primera conexión de la captura.

`tcp.stream` es muy útil para filtrar una conexión completa, pero **no viaja en el paquete**: lo calcula la herramienta al leer el archivo. Por eso, al citarlo, se acompaña de la 5-tupla o de los números de paquete.

Retransmisiones

Cuando un segmento no se confirma a tiempo, el emisor lo vuelve a enviar. Wireshark marca esos paquetes como `TCP Retransmission` y los agrupa en el filtro `tcp.analysis.flags` . Una retransmisión indica pérdida o demora en algún punto del camino; no es, por sí misma, un indicador de ataque. Sí advierte que el sensor pudo haber perdido paquetes.

5. Los protocolos de esta lección

DNS

DNS traduce nombres a direcciones. Casi toda comunicación hacia Internet empieza con una consulta DNS, lo que la convierte en un punto de partida natural del análisis.

Campo	Significado
Consulta (<code>dns.qry.name</code>)	Nombre que el equipo quiso resolver
Tipo <code>A</code>	Solicita una dirección IPv4
Respuesta (<code>dns.a</code>)	Dirección obtenida
Código de respuesta (<code>dns.flags.rcode</code>)	<code>0</code> sin error; <code>3</code> (<code>NXDOMAIN</code>) el nombre no existe
TTL	Segundos durante los cuales el equipo puede reutilizar la respuesta sin volver a consultar

Una consulta DNS demuestra que el equipo **quiso** contactar un nombre, no que lo contactó. La conexión se demuestra con los paquetes que siguen. Y el TTL explica por qué no siempre hay una consulta antes de cada conexión: el equipo puede estar reutilizando una respuesta anterior.

Algunas consultas son ruido habitual de las redes corporativas. Windows, por ejemplo, busca por defecto un nombre `wpad` para descubrir si existe un proxy configurado; una respuesta `NXDOMAIN` a esa consulta es normal y no forma parte de un incidente.

HTTP

HTTP sin cifrar transmite todo en texto legible: solicitudes, cabeceras y contenido.

Elemento	Ejemplo	Valor forense
Línea de solicitud	<code>GET /ruta/recurso HTTP/1.1</code>	Método y recurso solicitado
<code>Host</code>	Nombre del sitio	Qué sitio se pidió, aunque varios compartan la misma IP
<code>User-Agent</code>	Cadena que declara el cliente	Pista sobre el programa que hizo la solicitud; puede falsificarse
Código de estado	<code>200</code> , <code>404</code> , <code>500</code>	Resultado de la solicitud
<code>Content-Type</code>	<code>text/plain</code> , <code>text/html</code>	Tipo de contenido que declara el servidor
<code>Content-Length</code>	Número de bytes	Tamaño esperado del cuerpo; permite contrastar el objeto extraído
<code>Content-Disposition</code>	<code>attachment;</code> <code>filename="..."</code>	Nombre de archivo sugerido por el servidor

Los códigos se agrupan por su primer dígito: `2xx` éxito, `3xx` redirección, `4xx` error del cliente (`404` recurso inexistente) y `5xx` error del servidor.

Una conexión HTTP/1.1 puede transportar **varias solicitudes sucesivas** si ambas partes la mantienen abierta (`keep-alive`). Por eso una misma `tcp.stream` puede contener más de una descarga.

Todo lo que el servidor declara es declaración: el nombre, el tipo y el tamaño pueden no coincidir con el contenido real. Verificar el tipo real de un archivo se trabaja en la Lección 13; aquí basta con registrar lo declarado y calcular el hash.

TLS

TLS cifra el contenido de la comunicación. Es la base de HTTPS. Antes de cifrar, cliente y servidor negocian parámetros en un **handshake** que, en gran parte, viaja en claro.

Visible en la captura	No visible sin las claves
Direcciones IP, puertos, horas, tamaños y cantidad de paquetes	Rutas, cabeceras y contenido de las solicitudes
Nombre de servidor declarado por el cliente (SNI)	Archivos transferidos
Versión y algoritmos negociados	Credenciales o datos enviados
Duración y frecuencia de las conexiones	Qué programa del equipo inició la conexión

El **SNI** (*Server Name Indication*) es el nombre que el cliente declara en el `Client Hello` para que el servidor sepa qué sitio atender. Es uno de los datos más útiles del tráfico cifrado, con dos advertencias: lo escribe el cliente, así que puede no corresponder al servicio real, y tecnologías recientes como *Encrypted Client Hello* pueden ocultarlo.

Por compatibilidad, el `Client Hello` de TLS 1.3 declara una versión antigua en su cabecera y anuncia la versión real en una extensión. Por eso Wireshark puede rotular ese paquete como `TLSv1.2` aunque la sesión sea TLS 1.3.

Dos handshakes distintos: TCP y TLS

En una conexión HTTPS ocurren dos *handshakes*, uno después del otro, en capas diferentes. Se confunden porque ambos se llaman *handshake*, pero responden preguntas distintas.



Aspecto	Three-way handshake (TCP)	Handshake TLS
Capa	Transporte	Sobre TCP, antes de la aplicación
Propósito	Abrir la conexión: ambos extremos acuerdan números de secuencia	Negociar versión, algoritmos y claves para cifrar lo que viene

Aspecto	Three-way handshake (TCP)	Handshake TLS
Mensajes	Exactamente tres: SYN , SYN, ACK y ACK	Varios: comienza con Client Hello y Server Hello ; su cantidad depende de la versión
Contenido	Solo cabeceras TCP, sin datos	Datos de la negociación; en TLS 1.3, después del Server Hello el resto del handshake ya viaja cifrado
Qué deja ver	Que la conexión se estableció, entre qué puertos y cuándo	SNI, versión y algoritmos; en TLS 1.2 también el certificado del servidor
Filtro	tcp.flags.syn == 1 && tcp.flags.ack == 0 para los SYN	tls.handshake.type == 1 para los Client Hello y == 2 para los Server Hello
Aparece en	Toda conexión TCP: HTTP, HTTPS, SSH, correo	Solo en conexiones que usan TLS, como HTTPS

El orden importa: **sin three-way handshake completo no hay handshake TLS**, porque TLS necesita una conexión TCP ya abierta. Por eso cada combinación se lee distinto:

Lo que muestra la captura	Lectura forense
SYN sin respuesta, o SYN seguido de RST	No se estableció la conexión; tampoco hubo TLS
Three-way handshake completo sin Client Hello	Se abrió la conexión, pero no se inició TLS: otro protocolo en ese puerto, una prueba de conexión o una captura cortada
Client Hello sin Server Hello	El cliente pidió cifrar, pero el servidor no respondió o cerró la conexión: no hubo comunicación cifrada
Ambos handshakes completos y Application Data	Hubo comunicación cifrada; su contenido no es legible sin las claves

Una nota para más adelante: HTTP/3 usa QUIC, que viaja sobre UDP e integra TLS 1.3 en su propio handshake. En esas conexiones no hay three-way handshake TCP.

ARP y NTP

Aparecen en casi toda captura y sirven como contexto. **ARP** resuelve qué MAC corresponde a una IP dentro de la red local; muestra qué equipos estaban presentes en el segmento. **NTP** sincroniza relojes; su presencia indica que el equipo intentó ajustar su hora, un dato útil al comparar la captura con los registros del propio equipo.

6. Filtros: seleccionar sin alterar

Existen dos tipos de filtro que se confunden con frecuencia:

Tipo	Cuándo actúa	Sintaxis	Efecto
Filtro de captura	Mientras se captura	BPF: host 10.0.0.5 and port 80	Descarta paquetes para siempre: nunca llegan al archivo
Filtro de visualización	Al analizar un archivo ya capturado	Wireshark: ip.addr == 10.0.0.5 && tcp.port == 80	Oculta paquetes en pantalla; el archivo no cambia

En el análisis forense de una captura entregada solo se usan filtros de visualización. Un filtro de captura mal elegido durante la adquisición es una pérdida irreversible, y debe declararse en la ficha técnica si se usó.

Filtros frecuentes

Objetivo	Filtro de visualización
Todo el tráfico de una dirección	ip.addr == 10.0.0.5
Solo lo que envía una dirección	ip.src == 10.0.0.5
Una conexión TCP completa	tcp.stream eq 4
Consultas y respuestas DNS	dns
Solo respuestas DNS	dns.flags.response == 1
Nombres inexistentes	dns.flags.rcode == 3
Solicitudes HTTP	http.request
Respuestas HTTP con error	http.response.code >= 400

Objetivo	Filtro de visualización
Mensajes <code>Client Hello</code> del cliente	<code>tls.handshake.type == 1</code>
Un nombre de servidor concreto	<code>tls.handshake.extensions_server_name == "ejemplo.invalid"</code>
Observaciones del análisis TCP	<code>tcp.analysis.flags</code>

Se combinan con `&&` (y), `||` (o) y `!` (negación).

Reproducible o no reproducible

Un filtro es evidencia cuando otra persona puede escribirlo tal cual, sobre la misma captura, y obtener el mismo resultado. Por eso se registra el texto exacto del filtro y la cantidad de paquetes que produce. Una captura de pantalla sin el filtro visible no permite repetir el análisis.

Resolución de nombres desactivada

Wireshark y `tshark` pueden traducir direcciones a nombres consultando el DNS del equipo de análisis. En forense se desactiva: genera tráfico nuevo desde el laboratorio, puede alertar a quien controla el dominio y mezcla en el análisis nombres que se obtuvieron **hoy** con los que existían **en la captura**. Los nombres válidos son los que aparecen dentro de los paquetes, como las consultas DNS o el campo `Host` .

7. Estadísticas: mirar antes de buscar

Antes de buscar lo sospechoso conviene describir la captura completa. Tres vistas bastan:

Vista	Pregunta que responde	Error que evita
Jerarquía de protocolos	¿Qué protocolos contiene la captura y en qué proporción?	Buscar HTTP en una captura donde todo es TLS
Conversaciones	¿Qué pares de direcciones se comunicaron, cuánto y cuándo?	Pasar por alto el equipo o el destino con más tráfico
<i>Endpoints</i>	¿Qué direcciones participan y cuánto envió y recibió cada una?	Olvidar un equipo que solo aparece recibiendo

Esta mirada panorámica también revela el tráfico **normal** de la red, que servirá de comparación: si otro equipo de la misma oficina muestra un patrón parecido, el patrón deja de ser excepcional.

8. Reconstruir una sesión

Reconstruir una sesión es presentar, en orden y con sus números de paquete, todo lo que ocurrió entre un cliente y un servidor para cumplir una tarea.

1. **Partir del nombre.** Localizar la consulta y la respuesta DNS que llevan a la dirección de interés.
2. **Ubicar la conexión.** Filtrar por esa dirección y encontrar el `SYN` que abre la conexión; anotar el puerto efímero y el `tcp.stream` .
3. **Aislar la conexión.** Filtrar por `tcp.stream eq N` .
4. **Leer el intercambio.** Seguir el *stream* TCP (Follow TCP Stream) para ver solicitudes y respuestas como texto.
5. **Registrar cada paso.** Número de paquete, hora UTC y local, origen y destino con puertos, y el dato observado.
6. **Cerrar.** Anotar cómo terminó la conexión y qué quedó sin respuesta.

Seguir el *stream* presenta los bytes de ambos sentidos reunidos y en orden. Es lectura de la captura: no ejecuta nada. Sirve para leer protocolos en texto como HTTP; en TLS solo muestra bytes cifrados.

9. Extracción controlada de objetos

Cuando el contenido viaja sin cifrar, Wireshark puede reconstruir los archivos transferidos: **Export Objects** en la interfaz, o `--export-objects` en `tshark` . Extraer un objeto crea un **artefacto derivado** de la evidencia y exige el mismo cuidado que cualquier otra copia.

Regla	Razón
Exportar fuera de la carpeta de evidencia	La evidencia no se modifica ni se mezcla con derivados
Registrar el paquete de origen	El objeto debe poder rastrearse hasta la respuesta que lo contenía

Regla	Razón
Calcular SHA-256 inmediatamente	Identifica el objeto sin interpretarlo y permite compararlo después
Contrastar tamaño y <code>Content-Length</code>	Una diferencia indica una transferencia incompleta o una reconstrucción defectuosa
No abrir con aplicaciones asociadas ni ejecutar	Un objeto transferido puede ser dañino; el nombre y el tipo declarados no son garantía
No subir a servicios públicos de análisis	Publica evidencia del caso y puede alertar a quien preparó el archivo

La herramienta nombra los objetos a partir de la ruta solicitada o del nombre declarado por el servidor. Si varias respuestas se exportan a la vez, hay que identificar cuál corresponde a cada solicitud, por tamaño, tipo y paquete.

10. Patrones en el tiempo

Algunas conclusiones de red no provienen de un paquete, sino de la **repetición**. Un equipo que contacta el mismo destino a intervalos casi constantes presenta un patrón periódico. En el lenguaje de la seguridad, cuando se sospecha que un programa consulta regularmente a un servidor de control, ese patrón se llama *beaconing*.

Observación	Cómo se mide
Intervalo entre conexiones	Diferencia de tiempo entre <code>Client Hello</code> sucesivos al mismo destino
Regularidad	Cuánto varía ese intervalo; una variación pequeña se llama <i>jitter</i>
Volumen por conexión	Bytes enviados y recibidos; si son casi idénticos, el intercambio es automático y repetitivo
Duración	Cuánto dura cada conexión y cuántas se observan en la ventana

La periodicidad demuestra **automatización**, no intención maliciosa. Muchos programas legítimos se comportan igual: clientes de correo que sincronizan, agentes de actualización, telemetría del sistema operativo, antivirus o el propio NTP. Un patrón periódico es un motivo para preguntar qué proceso del equipo lo produce, no una respuesta.

11. Indicadores de compromiso

Un **indicador de compromiso** (IoC) es un dato observable que, en un contexto dado, se asocia con actividad no autorizada: una dirección IP, un dominio, una URL, el hash de un archivo, una cadena `User-Agent` , un patrón de conexiones.

Un indicador sin contexto no es un hallazgo

Elemento	Pregunta que responde
Indicador	¿Qué dato exacto se observó?
Fuente	¿En qué paquetes o registros aparece?
Ventana	¿Entre qué horas?
Razón de interés	¿Por qué merece atención en este caso?
Uso legítimo posible	¿Qué explicación inocente es compatible con el dato?

No todos los indicadores valen lo mismo

David Bianco propuso ordenar los indicadores según cuánto le cuesta al adversario cambiarlos, en lo que llamó la *Pirámide del dolor* (*Pyramid of Pain*). En la base están los hashes de archivo y las direcciones IP: basta recompilar un archivo o cambiar de servidor para invalidarlas. Más arriba están los dominios, luego los artefactos de red y de equipo, las herramientas y, en la cima, las técnicas. Un indicador de la base es fácil de obtener y fácil de evadir; uno de la cima es difícil de obtener y costoso de cambiar.

Reglas de manejo

- Neutralizar** (*defang*) dominios y URL al escribirlos, para que nadie los abra por accidente: `ejemplo[.]invalid` , `hxxp://` .
- No consultar** los indicadores en servicios públicos sin autorización: la consulta puede quedar registrada y alertar a quien controla la infraestructura.
- Registrar la ventana**: una dirección IP puede cambiar de dueño; un indicador de hace un año puede señalar hoy a un servicio legítimo.
- Declarar el falso positivo posible**: si no se puede imaginar ninguno, probablemente no se buscó.

12. Hecho, inferencia y límites de la red

Los ejemplos de esta tabla provienen del caso resuelto de la sección 14.

Nivel	Ejemplo
Dato observado	El paquete 5 contiene <code>GET /manual.txt</code> enviado desde <code>10.0.0.15</code>
Relación	La respuesta del paquete 6 pertenece a la misma conexión y declara 640 bytes
Inferencia	El equipo <code>10.0.0.15</code> descargó un archivo de texto desde ese servidor
Explicación alternativa	Un programa de actualización pudo solicitarlo sin intervención de una persona
Conclusión acotada	Dentro de la ventana capturada, el equipo recibió un archivo de 640 bytes; la captura no muestra qué hizo el equipo con él

Lo que una captura no puede mostrar

Límite	Formulación correcta
Persona	«El equipo <code>10.0.0.15</code> inició la conexión», no «el usuario descargó».
Programa	«El <code>User-Agent</code> declara X», no «lo descargó el programa X».
Ejecución	«El archivo se transfirió», no «el archivo se ejecutó».
Contenido cifrado	«Se intercambiaron N bytes cifrados con el servidor que declara el nombre Y», no «se enviaron datos robados».
Cobertura	«No se observó en este sensor y en esta ventana», no «no ocurrió».
Causalidad	«La serie comenzó después de la descarga», no «la descarga provocó la serie».

Las preguntas sobre programa, persona y ejecución se responden en el equipo: procesos creados, tareas programadas y Prefetch, como en la Lección 07. La red y el sistema operativo se complementan; ninguno reemplaza al otro.

13. Errores frecuentes

Error	Por qué ocurre	Corrección
Analizar con resolución de nombres activa	Es la configuración por defecto	Desactivarla antes de abrir la captura y declararlo
Citar una hora sin zona	Se copia lo que muestra Wireshark	Trabajar en UTC y normalizar explícitamente
Confundir consulta DNS con conexión	Se asume que resolver es contactar	Buscar el <i>handshake</i> TCP que sigue
Confundir el <i>three-way handshake</i> con el <i>handshake</i> TLS	Ambos se llaman <i>handshake</i>	TCP abre la conexión con tres mensajes; TLS negocia el cifrado después, sobre esa conexión
Tratar <code>tcp.stream</code> como un dato del paquete	La herramienta lo muestra como un campo más	Acompañarlo de la 5-tupla o de los números de paquete
Leer el <code>User-Agent</code> como prueba del programa	La cadena parece técnica	Tratarla como declaración del cliente
Abrir el objeto exportado para «ver qué es»	Curiosidad o apuro	Calcular el hash y leer el contenido solo en el <i>stream</i> de la captura
Declarar malicioso un dominio desconocido	Desconocido parece sospechoso	Buscar uso legítimo posible y comparar con otros equipos
Concluir intención desde la periodicidad	El patrón recuerda al malware	Declarar automatización y pedir evidencia del equipo
Afirmar lo que TLS oculta	Se completa desde la expectativa	Limitarse a SNI, tamaños, tiempos y frecuencia
Entregar capturas de pantalla sin filtro	Se prioriza lo visual	Registrar el filtro exacto y el conteo

14. Ejemplo resuelto - caso ALERCE LTDA (ficticio)

Se autoriza examinar una captura verificada del puerto SPAN de la oficina de ALERCE LTDA para responder:
¿qué obtuvo el equipo 10.0.0.15 desde el exterior entre las 14:00:00 y las 14:00:05 UTC del 3 de julio de 2026? La captura se tomó con resolución de nombres desactivada.

Sesión reconstruida

N.º	Hora UTC	Origen	Destino	Protocolo	Información
1	14:00:01.100	10.0.0.15	10.0.0.53	DNS	Consulta A files-lab.invalid
2	14:00:01.140	10.0.0.53	10.0.0.15	DNS	Respuesta A 192.0.2.60
3	14:00:02.000	10.0.0.15:51514	192.0.2.60:80	TCP	SYN
4	14:00:02.030	192.0.2.60:80	10.0.0.15:51514	TCP	SYN, ACK
5	14:00:02.090	10.0.0.15:51514	192.0.2.60:80	HTTP	GET /manual.txt
6	14:00:02.160	192.0.2.60:80	10.0.0.15:51514	HTTP	200 OK , 640 bytes
7	14:00:03.000	10.0.0.15:51514	192.0.2.60:80	TCP	FIN, ACK

Filtros usados

dns	-> paquetes 1 y 2
ip.addr == 192.0.2.60 && tcp.port == 80	-> paquetes 3 a 7
tcp.stream eq 0	-> paquetes 3 a 7

Indicadores

Indicador	Paquetes	Por qué interesa	Uso legítimo posible
files-lab[.]invalid	1-2	Dominio que precede a la descarga	Repositorio de documentación del proveedor
192.0.2.60	2-7	Servidor que entregó el archivo	Alojamiento compartido por muchos sitios
hxxp://files-lab[.]invalid/manual.txt	5	Recurso concreto transferido	Manual solicitado por un programa de ayuda

Conclusión de la etapa

Los paquetes 1 y 2 resuelven files-lab[.]invalid como 192.0.2.60 ; los paquetes 3 a 7 registran una conexión desde 10.0.0.15:51514 en la que se solicita /manual.txt y se recibe una respuesta 200 de 640 bytes. La captura demuestra la transferencia, pero no identifica a la persona ni el programa que la solicitó, ni muestra qué ocurrió con el archivo en el equipo. Se recomienda revisar los registros de procesos del equipo para el mismo minuto.

El texto informa hechos con sus paquetes, una relación y lo que falta. No califica el archivo ni atribuye intención.

15. Preparación para la actividad

Durante la práctica se trabajará sobre RED-CAS002 , extraída para un intervalo solicitado de diez minutos en una oficina ficticia. La ventana observable se determina con las marcas del primer y último paquete. Conviene llegar con estas respuestas claras:

1. ¿Qué demuestra una consulta DNS y qué no demuestra?
2. ¿Cómo se aísla una conexión completa en una captura?
3. ¿Por qué el User-Agent es una pista y no una prueba?
4. ¿Qué se puede leer de una conexión TLS sin las claves?
5. ¿Qué se hace con un objeto exportado antes de cualquier otra cosa?
6. ¿Por qué una serie de conexiones periódicas no demuestra por sí sola una infección?

16. Glosario

Término	Definición
5-tupla	IP y puerto de origen, IP y puerto de destino, y protocolo de una conexión.
Beaconing	Patrón de conexiones periódicas de un programa hacia un mismo servidor, estudiado como posible señal de control remoto.
Captura completa de paquetes	Registro de cada paquete con sus cabeceras y contenido.
Client Hello	Primer mensaje del <i>handshake</i> TLS. Lo envía el cliente en claro e incluye el SNI.
Conversación	Conjunto de paquetes intercambiados entre dos direcciones.
Filtro de visualización (<i>display filter</i>)	Expresión que selecciona paquetes de una captura ya realizada sin modificarla.
Handshake	Intercambio inicial con que dos equipos establecen una conexión (TCP) o negocian su cifrado (TLS).
Handshake TLS	Negociación de versión, algoritmos y claves que ocurre sobre una conexión TCP ya abierta; comienza con <code>Client Hello</code> .
Hash	Valor calculado a partir del contenido de un archivo, como SHA-256, que cambia si el contenido cambia.
Indicador de compromiso (IoC)	Dato observable que, en un contexto, se asocia con actividad no autorizada.
Jitter	Variación del intervalo entre eventos que deberían ser regulares.
Neutralizar (<i>defang</i>)	Escribir un dominio o una URL de modo que no sea un enlace activo.
OUI	Prefijo de tres bytes de una dirección MAC asignado a un fabricante.
Puerto efímero	Puerto alto y temporal que elige el cliente para una conexión.
Puerto SPAN (<i>port mirroring</i>)	Función de un switch que copia el tráfico de uno o más puertos hacia un sensor.
SNI	Nombre de servidor que el cliente declara en claro al iniciar TLS.
Snaplen	Cantidad máxima de bytes que se guardan de cada paquete.
Stream TCP	Todos los bytes de una conexión TCP, reunidos en orden y en ambos sentidos.
<code>tcp.stream</code>	Índice que Wireshark asigna a cada conexión TCP de una captura.
Three-way handshake	Apertura de una conexión TCP mediante tres mensajes: <code>SYN</code> , <code>SYN, ACK</code> y <code>ACK</code> .
Timestamp	Fecha y hora registradas para un paquete o evento; en una captura, las toma el reloj del sensor.

Fuentes

- [Wireshark User's Guide](#). Estadísticas, filtros, seguimiento de streams y exportación de objetos.
- [Wireshark Display Filter Reference](#). Nombres de campos de cada protocolo.
- [tshark manual](#). Opciones de línea de comandos usadas en la guía.
- [Wireshark OUI Lookup](#). Consulta del fabricante asociado a un prefijo MAC.
- [NIST SP 800-86 - Guide to Integrating Forensic Techniques into Incident Response](#). Fuentes de datos de red y su examen.
- [NIST SP 800-92 - Guide to Computer Security Log Management](#). Registros de red y su gestión.
- [RFC 1035 - Domain Names: Implementation and Specification](#). Estructura de consultas y respuestas DNS.
- [RFC 9110 - HTTP Semantics](#). Métodos, códigos de estado y cabeceras.
- [RFC 8446 - The Transport Layer Security \(TLS\) Protocol Version 1.3](#). *Handshake* y compatibilidad de versiones.
- [RFC 6066 - TLS Extensions](#). Extensión *Server Name Indication*.
- [RFC 5737 - IPv4 Address Blocks Reserved for Documentation](#). Rangos usados en los casos del curso.
- [David Bianco - The Pyramid of Pain](#). Valor relativo de los indicadores.